

Step 1 : When the server receives a request for authentication, such as a login request, it is first intercepted by a class - DelegatingFilterProxy
It's a Proxy for a standard Servlet Filter, delegating to a Spring-managed bean that implements the Filter interface.

It will delegate the request to Authentication Filter(eg. BasicAuthenticationFilter|UsernamePasswordAuthenticationFilter|Custom JWT Filter) in the Filter Chain.

class UsernamePasswordAuthenticationFilter

Processes an authentication form submission. Login forms must present two parameters to this filter: a username and password

Step 2 : A UsernamePasswordAuthenticationToken is created using the username and password provided by the user. The UsernamePasswordAuthenticationToken is an implementation of the Authentication interface and used when a user wants to authenticate using a username and password.

Step 3 : The authentication filter delegates the process of authentication to AuthenticationManager (i/f) Method of AuthenticationManager -
Authentication authenticate(Authentication authentication) throws AuthenticationException

I/P - Authentication object (UsernamePasswordAuthenticationToken) containing un verified credentials

O/P - Returns a fully populated Authentication object (including granted authorities) in case of success or throws AuthenticationException

Step 4 : The AuthenticationManager delegates the authentication to the appropriate AuthenticationProvider. This is done by calling the supports() method on the AuthenticationProvider.

Step 5 : The most common implementation of AuthenticationProvider i/f - DaoAuthenticationProvider class

It's dependencies - UserDetailsServiceImpl PasswordEncoder
(Already configured in earlier project !)

The DaoAuthenticationProvider calls the loadUserByUsername(username) method of the UserDetailsServiceImpl and gets back the UserDetails object containing all the data of the user(loaded from DB).

The most important data is the password because it will be used to check whether the provided password is correct. If no user is found with the given user name, a UsernameNotFoundException is thrown.

Step 6 : The AuthenticationProvider after receiving the UserDetails checks the passwords and authenticates the user. If the passwords do not match it throws a AuthenticationException. But, if the authentication is successful, a UsernamePasswordAuthenticationToken is created, and the fields principal, credentials, and authenticated are set to appropriate values .

Principal refers to your username or the UserDetails (typically UserDetails in case of DaoAuthenticationProvider)

Credentials refers to password .

Authenticated field is set to true.

This token is returned back to the AuthenticationManager.

Step 7: On successful authentication, the SecurityContext is updated with the details of the current

authenticated user. SecurityContext can be used in several parts of the app to check whether any user is currently authenticated and if so, what are the user's details.

SO THAT IS THE ENTIRE PROCESS !!!!!